

Amlogic NPU User Guide

版本: v0.1

发布日期: 2026-05-31

版权

© 2026 晶晨半导体（上海）股份有限公司（代表其自身及其关联公司，以下简称"Amlogic"）版权所有。未经 Amlogic 书面许可，不得以任何形式或通过任何手段对本文件的任何部分进行复制、传播、转录或翻译成任何语言。

商标

 Amlogic，及 Amlogic 的其他图标均为 Amlogic 公司的商标。所有其他商标和注册商标均为其各自所有人的财产。

免责声明

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

本产品不适用于医疗、救生或生命维持等用途，也不适用于可合理预见因 Amlogic 产品故障或失灵可能导致人身伤害、死亡、严重财产或环境损害的应用场景。

文中提供的电路图及其他与 Amlogic 产品相关的信息仅为典型应用说明，完整且足以用于生产设计的全部信息不一定会提供。Amlogic 已尽量确保本文件内容准确和及时，但 Amlogic 不对本文档所含信息的准确性或完整性作任何保证。

Amlogic 产品可能存在未被识别的漏洞，或可能支持具有已知局限性的既定安全标准或规范。Amlogic 对任何漏洞不承担任何责任。

使用相关材料可能需获得第三方或 Amlogic 的知识产权许可。本文档对归属于 Amlogic 或任何其他方的知识产权不授予任何明示或暗示的声明或保证。文中包含的任何第三方网站链接仅为方便用户而提供，Amlogic 不对该等网站及其任何行为（包括隐私惯例、可用性和内容）作出认可，也不承担任何责任。

在任何情况下，Amlogic 均不对与本文档相关、由本文档使用或无法使用所产生的任何索赔承担责任。

联系方式

- 公司网址: www.amlogic.cn
- 售前咨询: contact@amlogic.com
- 技术支持: support@amlogic.com

修订记录

版本	修改说明
v0.1 (2026-05-31)	初始版本

目录

1 Amlogic NPU简介

- 1.1 Amlogic NPU软件堆栈介绍
- 1.2 Amlogic NN ToolChain介绍
- 1.3 集成流程简介
- 1.4 集成步骤
- 1.5 硬件平台支持
- 1.6 关键字说明

2 开发环境准备

- 2.1 开发环境确认
 - 2.1.1 DDK版本确认
 - 2.1.2 系统环境确认
 - 2.1.3 平台类型确认
 - 2.1.4 集成方式选择
- 2.2 Toolkit安装
 - 2.2.1 安装简介
 - 2.2.2 Toolkit安装
 - 2.2.2.1 配置虚拟环境
 - 2.2.2.2 安装Toolkit
 - 2.2.3 Native C 编译环境安装

3 模型转换

- 3.1模型转换简介
- 3.2模型转换准备
 - 3.2.1 模型信息确认
 - 3.2.2量化数据集准备
- 3.3 模型转换
 - 3.3.1 AMLNN初始化以及释放
 - 3.3.2 模型加载
 - 3.3.3 模型转换参数配置
 - 3.3.4 模型编译
 - 3.3.5 导出adla模型
 - 3.3.6 模型转换示例
- 3.4 模型评估
 - 3.4.1模型推理
 - 3.4.2 模型性能评估
 - 3.4.3 模型精度评估
 - 3.4.4 NPU 利用率
- 3.5 板端C API推理
 - 3.5.1 流程简介

4 板端Python API推理

- 4.1 Python API流程简介
- 4.2 Python API说明
 - 4.2.1 AMLNNLite初始化
 - 4.2.2 init_runtime
 - 4.2.3 get_sdk_version
 - 4.2.4 load_model
 - 4.2.5 get_tensor_info
 - 4.2.6 inference
 - 4.2.7 uninit
 - 4.2.8 Python API 示例Demo

1 Amlogic NPU简介

Amlogic NPU 是由 Amlogic NN Team 推出的一套面向 AI 模型部署的软件解决方案，旨在为 Amlogic 开发人员及客户提供完整的边缘计算模型部署平台。覆盖从模型转换到端侧推理的全流程，帮助用户快速评估模型在 Amlogic NN 平台上的可行性与性能表现，缩短整体开发周期。

1.1 Amlogic NPU软件堆栈介绍

软件栈架构图如下：

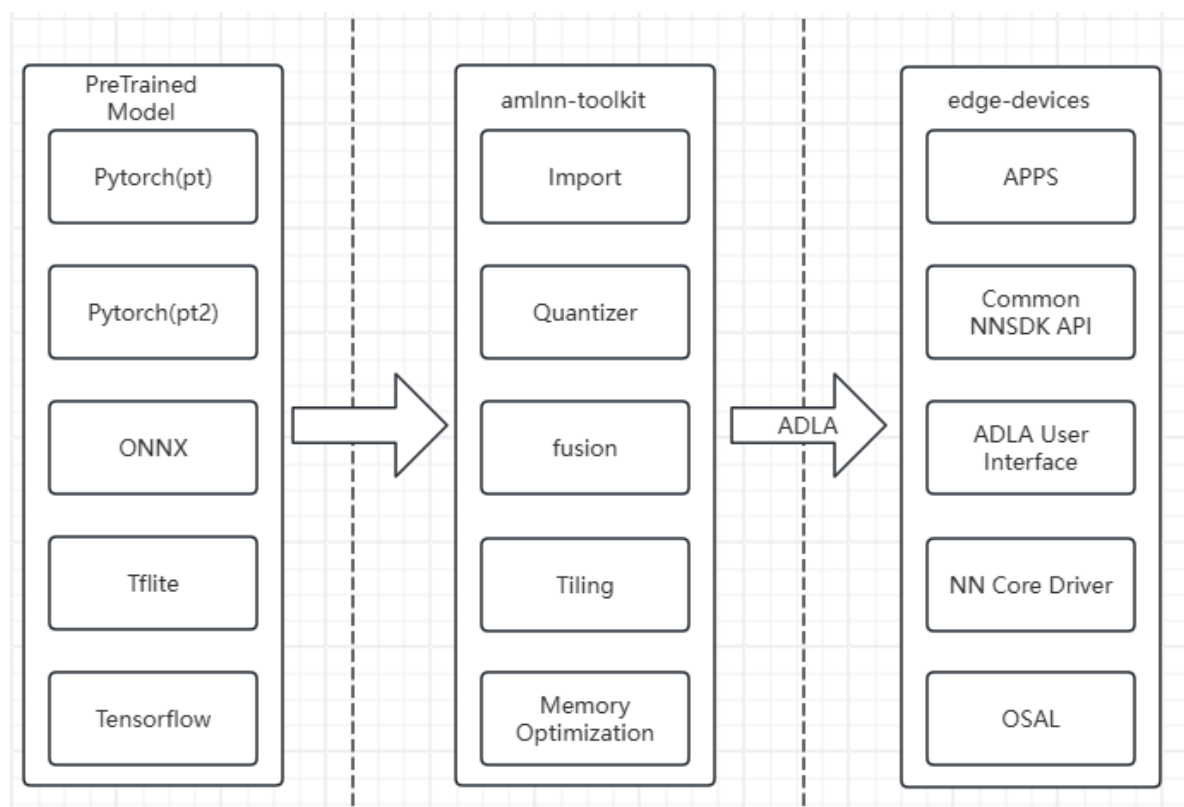


图1.1-NN_WorkFlow

1.2 Amlogic NN ToolChain介绍

- **ADLA Convert ToolChain:** 该组件支持在 Ubuntu PC 或板端 Debian 环境下，将 PyTorch、ONNX、TFLite 等框架模型导入并编译为 ADLA 模型文件，支持通过 ADB 连接板端或直接在 Debian 板端环境中，以 Python API 完成 ADLA 模型的加载与推理。当前 Release 包含四种 whl 包，对应说明如下：
 - **amlnn_toolkit**
 - Support Model quantization conversion, exporting ADLA on PC
 - Support ADLA model inferencing by Connect the board via ADB
 - Support GGUF(LLM/VLM) quantization conversion(参考LLM文档)
 - **amlnn_edge_toolkit**
 - Support Model quantization conversion, exporting ADLA on edge side
 - Support ADLA model inferencing by Connect the board via ADB
 - Support ADLA model inferencing on edge side

- **amlInn_edge_toolkit_lite**
 - Support ADLA model inferencing on edge side
- **amlIIm_edge_toolkit_lite**
 - Support ADLA model(LLM/VLM) inferencing on edge side (参考LLM文档)
- 三个whl包工具支持的功能差异对比如下表：

Function List	amlInn_toolkit	amlInn_edge_toolkit	amlInn_edge_toolkit_lite
Environment: System Env	x86_64	aarch64	aarch64
Conversion: Model quantization conversion & exporting ADLA	✓	✓	×
Conversion: Support GGUF(LLM/VLM) conversion & Exporting ADLA	✓	×	×
Inference: Support ADLA model inferencing via ADB	✓	✓	×
Inference: Support ADLA model inferencing on edge side	×	✓	✓

- **ADLA Runtime:** 面向 Amlogic NPU 的 C/C++ 推理 SDK，提供模型加载、推理执行、上下文管理等核心能力，支持快速集成与端侧高效部署。包含 `nn_runtime`（小模型）和 `llm_runtime`（大模型）两个运行时。
 - **nn_runtime:** 是面向 Amlogic NPU 平台的 C/C++ 推理 SDK，用于小模型部署。通过链接 `libnnsdk.so` 并包含 `nnsdk2.h` 头文件即可集成至原生应用，支持 Android 及 Linux (Buildroot / Yocto / Debian) 平台。
 - **llm_runtime:** 是面向 Amlogic NPU 平台的 C/C++ 推理 SDK，用于大语言模型部署。通过链接 `libllmsdk.so` 并包含 `llmsdk.h` 头文件即可集成至原生应用，支持 Android 及 Linux (Yocto / Debian) 平台。

1.3 集成流程简介

模型集成是指将在 PC 端 Python 环境下的模型加载与推理流程迁移至边缘设备上运行，并将前处理（PreProcess）、后处理（PostProcess）等操作改成 C/C++或java 实现。大致流程如下：

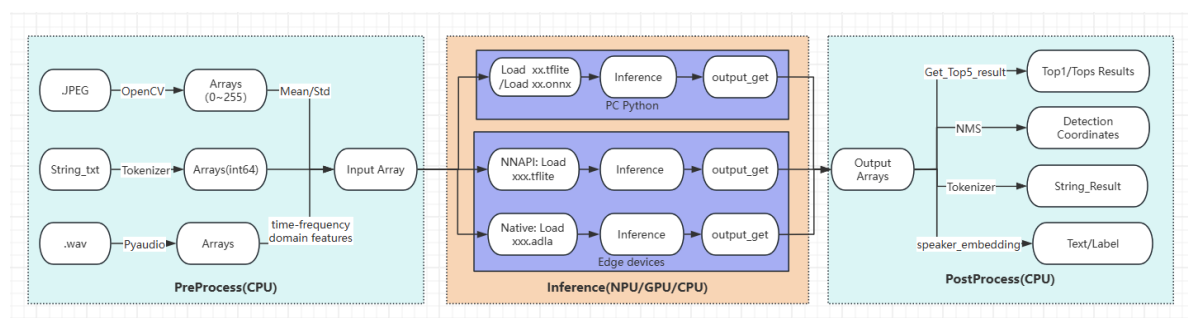


图1.3-集成流程

- **PreProcess:** 在边缘设备上集成模型时，需要将前处理模块用 C/C++或java实现。典型的前处理流程介绍如下：
 - **CV类模型前处理:** 以图片为输入，前处理模块包括：
 - **获取原始数据:** opencv读取图片成[0~255]范围的多维数组，或是从camera获取的多维数组
 - **归一化:** 根据模型训练时使用的均值(mean)和标准差(std)，对原始数据做归一化处理。对输入图像 x 的每个通道执行： $x(\text{norm})=(x-\text{mean})/\text{std}$ 。上述方法直接对取值范围为 [0, 255] 的整数像素值进行运算，这与 amlnn-toolkit 中读取图片并做量化时的处理方式一致。需要注意的是，某些框架会先将像素从 [0, 255] 缩放到 [0, 1]（即 $x / 255.0$ ），然后再执行减均值、除以标准差的标准化操作。使用时，请根据具体框架的要求自行换算。
 - **输入量化:** 仅当在边缘设备部署且生成的 ADLA 模型要求定点输入时，才需对归一化后的数据执行量化操作。量化依据 ADLA 模型的输入量化参数 (scale, zero_point) 进行： $x(\text{quant})=x(\text{norm})/\text{scale} + \text{zero_point}$
 - **前处理优化:** 在边缘设备部署时，可将归一化与输入量化的公式合并展开，从而减少冗余计算，有效降低前处理耗时。
 - **LLM Chat模型前处理:**
 - 使用 Tokenizer 将原始字符串转换为模型可接受的 input_ids 序列，同时可生成 attention_mask 等辅助输入。
 - **语音类模型前处理:**
 - 读取 .wav 文件并将其转换为数值数组，进而提取时频域特征。
- **Inference:**
 - **Native方式:**
 - **Native(c/c++):** 将 PC 端 ONNX、PyTorch、TFLite 等框架模型转换为 ADLA 模型文件后，通过 NNSDK 接口加载至 Amlogic NPU 进行推理运行。
 - **Native(Python Api):** 通过amlnn_edge_toolkit的Python API在板端做模型转换和集成，或将PC端转好的ADLA模型文件，通过amlnn_edge_toolkit_lite的Python API在板端做集成。
 - **Runtime方式:** 在 Android 系统中，通过 NNAPI 或 LiteRT 框架加载 TFLite 模型，并启用神经网络 (NN) 加速，最终使模型运行于 Amlogic NPU 上。
 - 加载的 TFLite 模型是量化版本还是浮点版本，取决于 Amlogic NPU 所支持的模型格式，以及对 NNAPI、LiteRT 等推理流程的适配情况。
 - **Third-Party Runtime方式:** 基于Amlogic已适配支持的第三方Runtime开源库加载运行。如：ONNXRuntime
- **PostProcess:** 在边缘设备上集成模型时，需要将后处理模块用 C/C++ 或java实现。典型的后处理流程介绍如下：
 - **分类模型后处理:** 对模型输出结果进行 Top-1 或 Top-5 处理，以获取对应的分类类别。
 - **检测类模型后处理:** 对模型输出的边界框和类别置信度进行解码、阈值过滤及非极大值抑制 (NMS)，得到最终的检测结果。
 - **LLM Chat模型后处理:** 将模型生成的 token 序列解码为自然语言文本，并去除填充或终止符等特殊标记。
 - **语音类模型后处理:** 对模型输出的声学特征或概率序列进行解码、去冗余及波形重构，得到最终的音频或识别文本。

1.4 集成步骤

整个集成流程大致分为以下几个步骤：

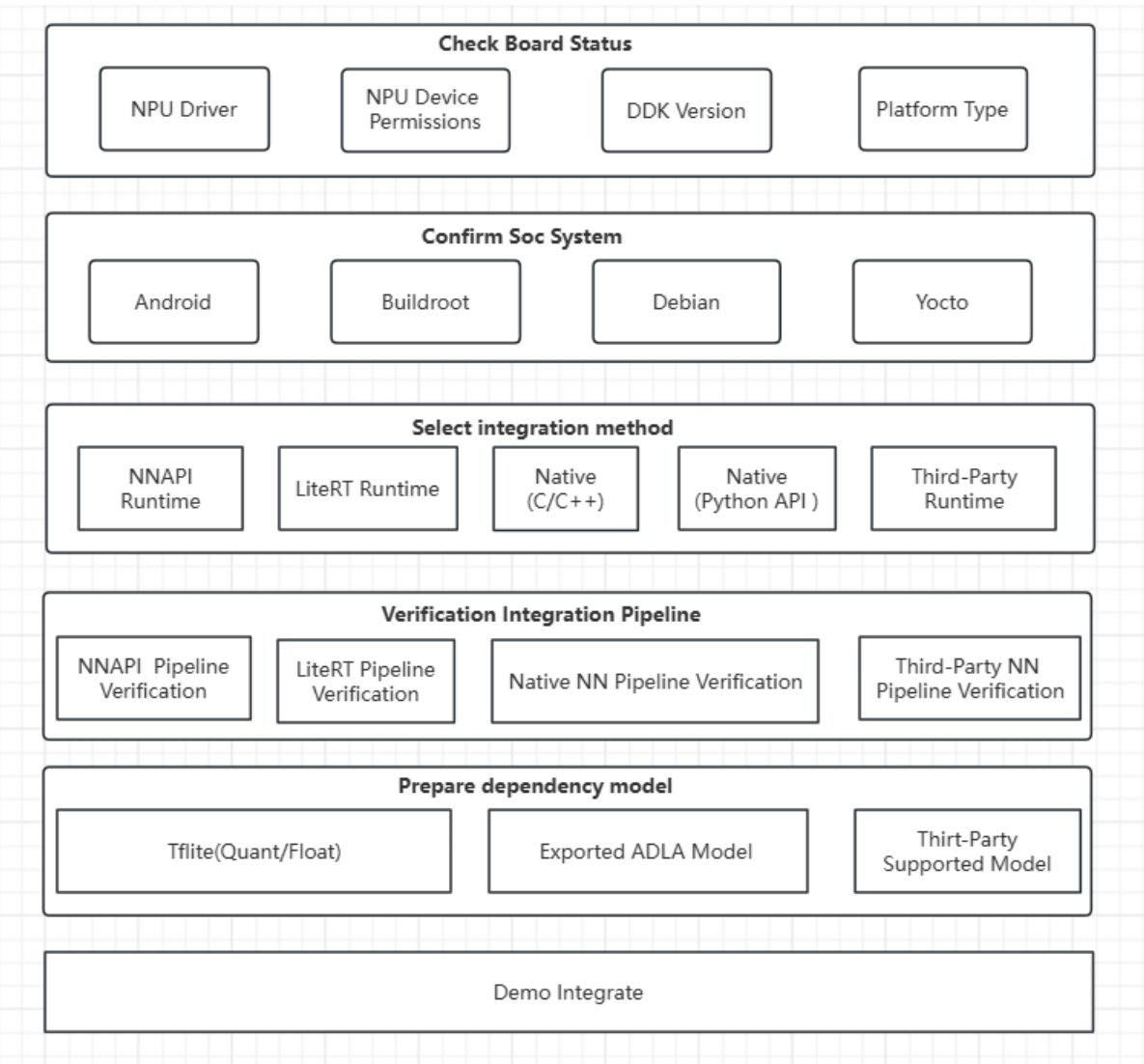


图1.4- 集成步骤

1. 开发环境确认：
- 确认边缘设备上的NN Driver版本号，并根据NN Driver版本确认DDK版本号

○ 确认边缘设备的系统环境[android/Buildroot /Debian/Yocto]
2. 集成方式确认：根据边缘设备的系统环境，选择集成方式。不同系统环境支持的集成方式如下：

	Buildroot	Android	Debian	Yocto	Dependency
Native(c/c++ Api)	✓	✓	✓	✓	amlInn_toolkit /ADLA NNSDK
Native(python Api)	×	×	✓	✓	amlInn_edge_toolkit / amlInn_edge_toolkit_lite
NNAPI Runtime	×	✓	×	×	Quant Tflite/Float Tflite
LiteRT Runtime	✓	✓	✓	✓	Quant Tflite/Float Tflite/Litert

	Buildroot	Android	Debian	Yocto	Dependency
Third-Party Runtime	✓	✓	✓	✓	ONNXRuntime

3. **集成方式Pipeline验证**：快速验证所选集成 Pipeline 的可行性
4. **模型选择**：选取 amlnn-toolkit 支持的开源框架模型，导出可独立部署的模型文件（含数据流图与权重，不依赖原始工程，可直接由 ONNX Runtime / TFLite Runtime / PyTorch 等框架加载运行）。
5. **模型转换**：
 - Native 集成：将开源框架模型转换为目标平台对应的 ADLA 模型。
 - NNAPI/LiteRT Runtime集成方式：将开源框架模型转换为 Quant TFLite 或 Float TFLite 模型。
6. **模型评估**：
 - Native 集成：评估转换后 ADLA 模型的精度是否满足需求。
 - NNAPI / LiteRT Runtime 集成：评估转换后 TFLite 模型的精度是否达标。
7. **Demo集成**：基于所选集成方式与开发环境，完成模型前处理、后处理开发，打通整条 Demo Pipeline。

1.5 硬件平台支持

本文档适用于如下 Amlogic 硬件平台：

- C308L / C302X
- S928X
- A311D2
- T968D4
- S905X5 / S905D5
- C302X2
- C305X2/A123X
- A311Y3

1.6 关键字说明

- **ADLA模型**：ADLA(amlogic Deep Learning Accelerator)，运行在amlogic 平台上的模型文件，后缀为.adla。
- **AMLNN**： Amlogic NPU
- **Edge Devices**：端侧边缘设备，在本文档中特指带有NPU的Amlogic单板。
- **W4A8/W8A8/W4A16/W8A16/W16A16**：以W4A8为例：weights 4bit，feature 8bit
- **Native方式**：不依赖第三方推理框架，且需提前将开源框架模型转成ADLA模型文件。
- **Runtime方式**：依赖第三方推理框架，无需提前转好ADLA模型文件，程序运行过程中转出ADLA模型文件。

2 开发环境准备

2.1 开发环境确认

在开展NN集成之前，需要先确认当前的单板环境，再选择集成方式，在1.4章节中已介绍需要的一些需要确认的信息。

2.1.1 DDK版本确认

需要先确认NPU模块是否加载OK，并确认加载的 NPU Driver 版本

- **Driver加载确认：**adb shell 连接单板或直接基于单板命令窗口
 - **加载确认：**执行：lsmod |grep adla_core 结果中有 adla_core，如下图：

```
t7 an400:/ $ lsmod |grep adla_core
adla_core                221184  0
amlogic_cpuinto          16384  18 amvdec_av1_t5d_v41,amvdec_
```

- **权限确认：**执行命令：ls /dev/adla0 -l 显示权限为666，如下图：

```
t7 an400:/ $ ls /dev/adla0 -la
crw-rw-rw- 1 system system 463,  0 1970-01-01 00:00 /dev/adla0
```

- **DDK版本确认：**
 - **Driver版本确认：**重启单板，在串口或者在CMD窗口adb shell连接后，执行命令：dmesg |grep adla，获取到类似如下log：

```
5.405204] adla kmd version: 0.10.7.1
```

NPU Driver 版本为：0.10.7 (获取前三位对应的版本号即可)

- **C Runtime版本确认：**板端支持如下确认命令

```
# Android 32 执行命令
strings /vendor/lib/libnnsdk.so | grep "NNSDK2 Version"

# Android 64 执行命令
strings /vendor/lib64/libnnsdk.so | grep "NNSDK2 Version"

# Linux (Buildroot 32 / Buildroot 64 / Yocto 32 / Yocto 64 / Debian 64 /
Ubuntu 64) 执行命令
strings /usr/lib/libnnsdk.so | grep "NNSDK2 Version"

# 打印如下，版本为 vx.x.x
NNSDK2 Version: vx.x.x, 20xx.xx
```

- **DDK版本确认：**

需根据 NPU Driver 版本号确认对应的 DDK 版本，同时确保 C Runtime 及 amlnn-toolkit 版本匹配。

DDK版本	NPU Driver 版本	C Runtime Version	amlnn-toolkit version
DDK4.7.9.8	1.7.1	2.8.5	adla-toolkit_3.2.9.3
DDK5.10.4.10	2.0.0		amlnn_toolkit

Note：当前文档内容支持NPU Driver版本>1.7.1以上的版本。

2.1.2 系统环境确认

确认板端系统环境与架构，旨在为后续章节选取对应系统的nnserver 及交叉编译工具链提供指导

Step1 先确认是Android平台或者Linux平台

执行命令：adb shell getprop ro.product.cpu.abi

判定逻辑：

- 输出 arm64-v8a：则确认为 64 位 Android 系统
- 输出 armeabi-v7a：则确认为32 位 Android 系统
- 报错 command not found：判定为 Linux 系统，请继续执行 Step 2

Step2 Linux 系统 (Buildroot / Yocto / Armbian) 环境进一步确认

Step2.1 确认是否为 Armbian / Yocto 系统

执行命令：cat /version.txt

参考输出示例(以 Armbian 为例)

```
imagename:a9-by401-bookworm-20260707030949
BRANCH=
VERSION=Linux/Debian/Armbian-PB#786
SPIN=Linux/Debian/Armbian-PB#786
BUILD_TIME="2026-07-07 03:27:41"
Generated on Tue Jul 7 03:27:41 UTC 2026

Compiler:
aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 11.3.Rel1) 11.3.1 20220712
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

判定逻辑：

- 有该文件存在:则判断为armbian/Yocto系统，再通过Compiler版本信息判断是32位/64位系统。参考示例对应Armbian 64-bit (AArch64)系统
- 文件不存在: 命令执行失败，判断为Buildroot系统，请继续执行Step2.2

注意：上述确认方式仅适用于 Amlogic 官方 Release 版本单板。第三方开发板请自行确认是否支持相同的版本文件与判定逻辑。

Step2.2 确认 Buildroot 系统架构

执行命令：adb shell uname -m

判定逻辑：

- aarch64 : 则确认为 64 位 buildroot系统
- armv7l : 则确认为 32 位 buildroot系统

2.1.3 平台类型确认

在启动ADLA模型转换流程前，必须先完成单板型号确认，获取与硬件完全匹配的 target_platform 参数，避免因平台不兼容导致转换失败或推理异常。当前通过以下两种独立方式完成校验，所有校验结果均可对照下面《Part_number/Serial/target_platform对应关系表》查询得到最终的 target_platform 值。对应关系表如下：

Part_number	Serial	target_platform
C308L / C302X	Serial: 3d0a010100000000c61349	PRODUCT_PID0XA001
S928X	Serial: 3e0a0202000000007c404b	PRODUCT_PID0XA002
A311D2	Serial: 360c01030000000025f904	PRODUCT_PID0XA003
T968D4	Serial: 420a010100000000daff6e	PRODUCT_PID0XA004
S905X5 / S905D5	Serial: 480a020100000000682763	PRODUCT_PID0XA005
C302X2	Serial: 4c0a01030000000055665a	PRODUCT_PID0XA006
A311Y3	Serial: 4d0a01020000000070e84a	PRODUCT_PID0XA007
C305X2/A123X	Serial: 4f0a01010000000054c6d1	PRODUCT_PID0XA008

方式一：通过 Part_number 快速确认

直接读取单板硬件表面刻印的Part_number标识，将读取到的编号与对应关系表匹配，即可获取该单板对应的正确 target_platform 参数。

方式二：通过 Serial 号校验确认

获取 Serial 信息：单板正常启动后，通过 ADB 连接设备，执行以下命令可读取 Serial 号：

```
adb shell cat /proc/cpuinfo
# 或
adb shell cat /proc/cpu_chipid
```

匹配规则：

- 首选规则：优先截取Serial号的前4位字符，对照对应关系表完成匹配，获取目标 target_platform 值。
- 备选规则：若前4位无匹配结果，可降级截取Serial号的前3位字符完成匹配，最终得到正确的 target_platform 参数。

不同单板支持的系统环境以及NPU硬件Feature情况如下表格：

	Buildroot	Android	Debian	Yocto	INT4	INT8	INT16	F16	BF16
--	-----------	---------	--------	-------	------	------	-------	-----	------

	Buildroot	Android	Debian	Yocto	INT4	INT8	INT16	F16	BF16
C308L/C302X	✓	×	×	×	×	✓	✓	×	×
S928X	×	✓	×	×	×	✓	✓	×	×
A311D2	×	✓	×	×	×	✓	✓	×	×
T968D4	×	✓	×	×	×	✓	✓	×	×
S905X5/S905D5	×	✓	×	✓	×	✓	✓	×	×
C302X2	✓	×	×	×	×	✓	✓	×	×
A311Y3	×	✓	✓	✓	✓	✓	✓	✓	✓
C305X2/A123X	✓	×	×	×	✓	✓	✓	✓	✓

2.1.4 集成方式选择

- 支持的集成方式：

	Buildroot	Android	Debian	Yocto	Dependency
Native(c/c++ Api)	✓	✓	✓	✓	amlInn_toolkit /ADLA NNSDK
Native(python Api)	×	×	✓	✓	amlInn_edge_toolkit / amlInn_edge_toolkit_lite
NNAPI Runtime	×	✓	×	×	Quant Tflite/Float Tflite
LiteRT Runtime	✓	✓	✓	✓	Quant Tflite/Float Tflite/Litert
Third-Party Runtime	✓	✓	✓	✓	ONNXRuntime

- 集成方式介绍

- Native(c/c++ Api)

- 使用toolkit将支持的开源框架模型转换导出成ADLA模型。
- Amlogic集成封装的统一NNSDK接口，实现ADLA模型的加载和推理。

- Native (Python Api)

- Method1：直接使用amlInn_edge_toolkit 在边缘设备端完成 ADLA 模型的转换与加载推理
- Method2：
 - 先使用toolkit将开源框架模型转换导出成ADLA模型。
 - 再使用amlInn_edge_toolkit或amlInn_edge_toolkit_lite的Python API实现ADLA模型的加载和推理。

- NNAPI Runtime

- 文档简介： [Neural Networks API](#) | [Android NDK](#) | [Android Developers](#)
- 模型准备：提前准备好 Quant / Float Tflite 模型。可借助 amlInn_toolkit（含 amlInn_edge_toolkit）将其他开源框架模型转换为 Quant Tflite 或 Float Tflite。
- 平台支持：当前 仅 A311Y3 同时支持 Quant Tflite + Float Tflite；其余平台仅支持 Quant Tflite。
- 支持状态：各个Android系统版本NNAPI支持状态如下：

Android System	NNAPI
Android R(11) 32bit/64bit	✓
Android T(13) 32bit/64bit	✓
Android U(14) 32bit/64bit	✓
Android B(16) 32bit/64bit	✓

◦ **LiteRT Runtime**

- 四个 SOC 系统环境均支持 LiteRT Runtime，当前除 Android 外，其余系统环境的使用指导待输出。

◦ **Third-Party Runtime**

- 搭建amlogic已支持适配的Third-Party 开源库环境
- 基于Third-Party环境，完成对支持模型的加载和推理（运行在Amlogic NPU）
- **Third-Party Runtime支持状态：**
 - 当前仅支持ONNXRuntime，其他Third-Part Runtime陆续支持中。

◦ **Android System: NNAPI Runtime VS Native(c/c++)**

	优点	缺点
Native	1. 性能最佳：直接使用NPU的接口可以充分利用特定硬件的能力，通常能获得更高的性能和更低的延迟。 2. 优化灵活：开发者可以针对特定的算法和模型进行深度优化，利用NPU的特性提供更高效的实现。 3. 性能调优：直接访问NPU可以让开发者实现低级别的控制，有助于进行特定的性能调优。	1. 开发复杂性：直接操作NPU的API，增加了开发和维护的复杂性。 2. 平台依赖性：不同厂商的NPU具有不同的API，可能导致代码不能跨设备复用，增加了适配多个平台的成本。
NNAPI	1. 跨平台兼容性：NNAPI提供了一个统一的接口，开发者可以在不同硬件和设备上调用同一个量化tflite。 2. 简易开发流程：开发者不用与底层硬件打交道，只需要转出量化tflite即可，只专注于模型的设计和应用开发。	1. 性能限制：NNAPI调用流程比较长，最终会在底层转出NPU可加载运行的adla文件，第一次调用时，有在线编译时间，耗时长。调用过程中消耗的内存更大。 2. 算子限制：并不是所有的操作和模型都能通过NNAPI得到支持，可能需要对模型进行调整才能适应NNAPI的要求 3. 潜在兼容性问题：同Android版本上NNAPI支持的算子存在差异，可能导致表现不一致。

2.2 Toolkit安装

2.2.1 安装简介

不同单板系统环境支持不同的集成方式，不同集成环境需要安装的Tool chain不一样。先根据系统环境确定集成方式，再安装对应的Toolkit。

- **Android System:**

Android	amlInn_toolkit	Native C编译环境
Native(c/c++ Api)	✓	✓
NNAPI Runtime	✓(可选)	×
LiteRT Runtime	✓(可选)	×
Third-Party Runtime	×	×

- **NNAPI/LiteRT Runtime:** amlInn_toolkit 为可选工具，用于将其他 Framework 模型转换为量化或浮点类型的 TFLite 模型。

- **Buildroot System:**

Buildroot	amlInn_toolkit	Native C编译环境
Native(c/c++ Api)	✓	✓
LiteRT Runtime	✓(可选)	×
Third-Party Runtime	×	×

- LiteRT Runtime**: amlInn_toolkit 为可选工具，用于将其他 Framework 模型转换为量化或浮点类型的 TFLite 模型。

- **Debian/Yocto System:**

Debian	amlInn_toolkit	amlInn_edge_toolkit	amlInn_edge_toolkit_lite	Native C编译环境
Native(c/c++ Api)	✓(可选)	✓(可选)	×	✓
Native(python Api)	✓(可选)	✓(可选)	✓(可选)	×
LiteRT Runtime	✓(可选)	✓(可选)	×	×
Third-Party Runtime	×	×	×	×

- **Native(c/c++ Api) :** amlInn_toolkit 或 amlInn_edge_toolkit 二选一，即可完成支持的 Framework 模型到 ADLA 模型的转换。
- **Native(python Api):**
 - Method1: 直接使用 amlInn_edge_toolkit 在边缘设备端完成 ADLA 模型的转换与加载推理。但受限于设备端内存，部分模型可能转换失败。
 - Method2:
 - ADLA模型转换: amlInn_toolkit 或 amlInn_edge_toolkit 二选一，即可完成支持的 Framework 模型到 ADLA 模型的转换。

- ADLA模型推理：amlInn_edge_toolkit或amlInn_edge_toolkit_lite 二选一，用于完成 ADLA 模型的加载和推理。
- **LiteRT Runtime:** amlInn_toolkit/amlInn_edge_toolkit 为可选工具，用于将其他 Framework 模型转换为量化或浮点类型的 TFLite 模型。

2.2.2 Toolkit安装

选定集成方式后，如需安装 Toolkit，可参考本章安装说明。PC 端及 Debian 设备端均推荐使用 Miniforge 管理 Python 环境。

2.2.2.1 配置虚拟环境

- **安装包下载：**
 - x86_64: wget https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh
 - aarch64: wget <https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh>
- **Miniforge安装：**
 - chmod +x Miniforge3-Linux-*.sh
 - X86_64: bash Miniforge3-Linux-x86_64.sh
 - aarch64: bash Miniforge3-Linux-aarch64.sh
- **验证安装：**
 - conda config --show channels 执行成功，结果中有 - conda-forge
- **虚拟环境创建**
 - 创建虚拟环境：conda create -n amlInnn_toolkit python=3.10
 - 进入虚拟环境：conda activate amlInnn_toolkit
 - Python 版本可按需调整（推荐 3.10）、'amlInn_toolkit' 为虚拟环境名称，可自行修改

2.2.2.2 安装Toolkit

激活amlInn_toolkit Conda环境之后，通过本地wheel包安装。

Install:

根据 Python 版本及处理器架构，选择对应的 requirements 和 whl 文件。

#for amlInn_toolkit

Step1:逐条安装依赖（避免版本冲突）

```
for req in $(cat requirements_x86_64.txt); do pip install $req; done
```

Step2:安装 toolkit whl 包

```
# toolkit-x.x.x 为工具包版本号, cpxx 为 Python 版本标签（如 cp310、cp311）
pip install amlInn_toolkit-x.x.x-cpxxx-cpxxx-linux_x86_64.whl
```

#Note: 请勿使用 pip install -r requirements_x86_64.txt，会导致依赖版本冲突。必须逐条安装。

#for amlInn_edge_toolkit

Step1:逐条安装依赖（避免版本冲突）

```
for req in $(cat requirements_aarch64.txt); do pip install $req; done
```

Step2:安装 toolkit whl 包

```
# toolkit-x.x.x 为工具包版本号, cpxx 为 Python 版本标签（如 cp310、cp311）
```

```
pip install amlnn_edge_toolkit-x.x.x-cpxxx-cpxxx-linux_aarch64.whl

#for amlnn_edge_toolkit_lite
pip install amlnn_edge_toolkit_lite-x.x.x-cpxxx-cpxxx-linux_aarch64.whl
```

Verify:

```
#验证toolkit-x.x.x版本
python -c "from amlnn.api import AMLNN; print('amlnn_toolkit import OK')"
```

```
#验证toolkit_lite-x.x.x版本
python -c "from amlnnlite.api import AMLNNLite; print('amlnn_edge_toolkit_lite
import OK')"
```

2.2.3 Native C 编译环境安装

Native c编译环境安装是在PC端安装编译设备系统对应的交叉编译工具。不同单板系统环境需要安装不同的工具链，具体如下：

- **Android System:**

- 交叉编译工具下载链接: [Andoird NDK下载地址](#)，推荐使用 android-ndk-r25c-linux.zip。
- 解压到指定目录,如: /opt/android-ndk-r25c
- 编译配置
 - 编译时, 在 `build-android64.sh` / `build-android32.sh` 中将 `ANDROID_NDK_PATH` 指定为本地 toolchain 路径。

- **Buildroot System:**

- 交叉编译工具下载链接:
 - 32bit: [Buildroot 32 toolchain下载地址](#)
 - 64bit: [Buildroot 64 toolchain下载地址](#)
- 解压到指定目录, 如:
 - 32bit: /opt/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi
 - 64bit: /opt/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu
- 编译配置
 - 编译时, 在 `build-buildroot64.sh` / `build-buildroot32.sh` 中将 `BUILDROOT_TOOLCHAIN_PATH` 指定为本地 toolchain 路径。

- **Debian/Yocto System:**

- 交叉编译工具下载链接:
 - 32bit: [32 toolchain下载地址](#)
 - 64bit: [64 toolchain下载地址](#)
- 执行对应脚本:
 - 32bit: `./poky-glibc-x86_64-amlogic-bsp-armv7at2hf-neon-mesons7-bh201-5.15-a32-toolchain-4.0.20.sh`
 - 64bit: `./poky-glibc-x86_64-meta-toolchain-armv8a-mesont7-an400-5.15-a64-toolchain-4.0.20.sh`
- 安装选择:

- 选择安装路径，如: /opt/poky/4.0.20。
- 选择输入 yes。
- 稍等片刻后，编译工具链即可安装完毕。
- 编译配置
 - 编译时，在 `build-yocto64.sh` / `build-yocto32.sh` 中将 `YOCTO_TOOLCHAIN_PATH` 指定为本地 toolchain 路径。

3 模型转换

3.1模型转换简介

- AMLNN模型转换流程图如下：

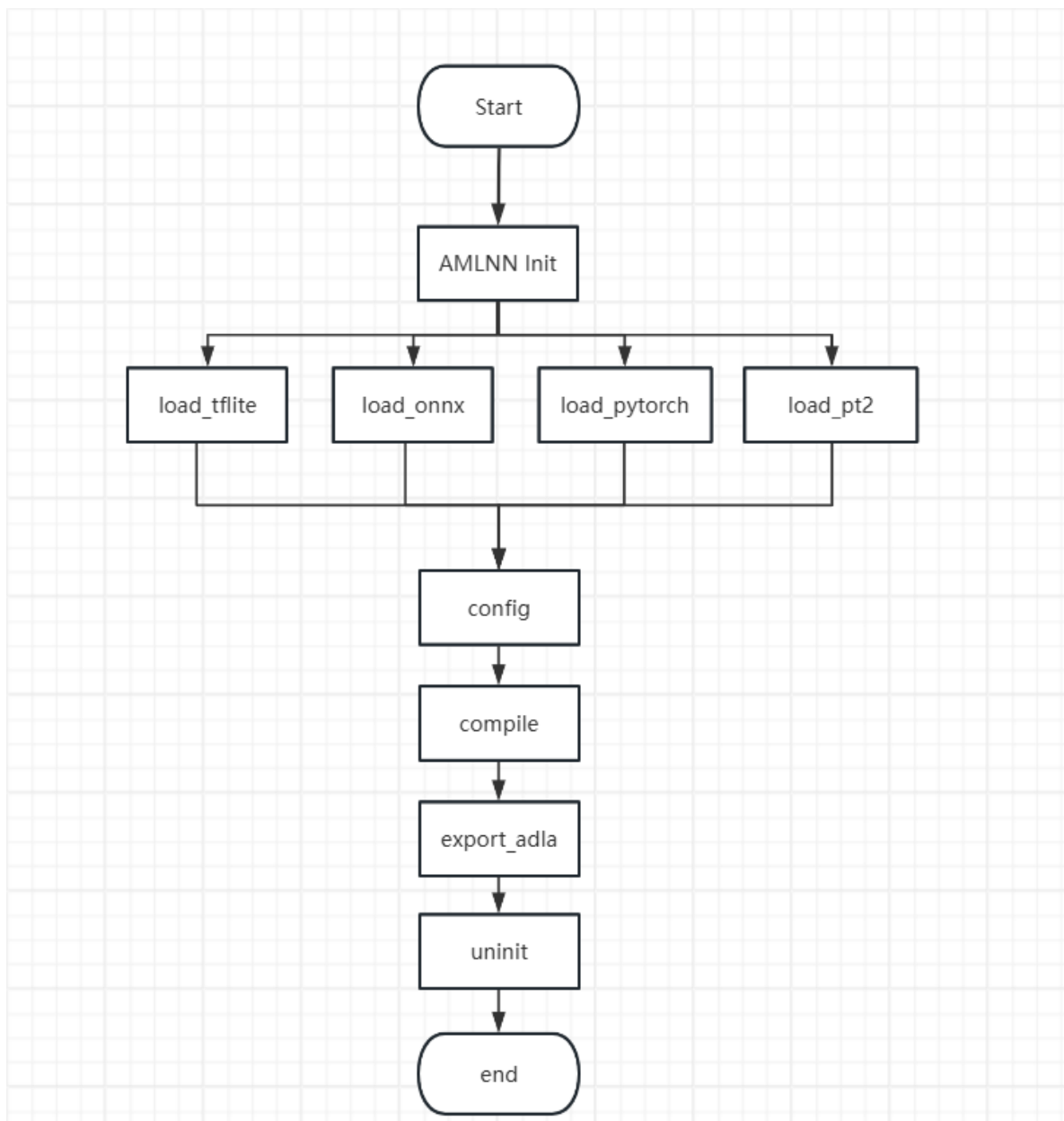


图3.1-模型转换流程图

- Amlnn-toolkit支持的framework版本如下：

Framework	Version	Note
Tflite	1.15~2.8.0	
ONNX	1.9.1	
Pytorch(jit)	>=2.6.0	Torchscript Format (torch.jit.trace、model.save)

Framework	Version	Note
Pytorch(pt2)	>=2.6.0	Exported_program Format(torch.export.save(exported_program, "model.pt2"))
Tensorflow	TODO	
Paddle	TODO	

3.2模型转换准备

3.2.1 模型信息确认

- 模型可视化工具

可以使用netron工具可视化模型，查看模型的输入输出信息以及模型结构。

- Netron工具对应的连接：
 - 网页版本: <https://netron.app/>
 - PC版本: <https://sourceforge.net/projects/netron.mirror/files/v8.0.4/>，下载Netron-Setup-8.0.4.exe文件直接双击安装。

- 模型可视化:

适合做独立模型需要有数据流+权重。以pytorch模型为例，一般会出现三种情况，如下图:

- 左边为traced_model = torch.jit.trace() /traced_model.save('model.pt') 保存的模型文件。有权重和数据流（箭头），支持独立部署。
- 中间为torch.save(model, 'model.pth') 保存的模型文件。这种模型对代码结构极度敏感大多数不支持独立部署，不推荐使用。
- 右边为torch.save(model.state_dict(), 'weights.pth')保存的模型文件。只有权重，没有数据流。不支持独立部署。

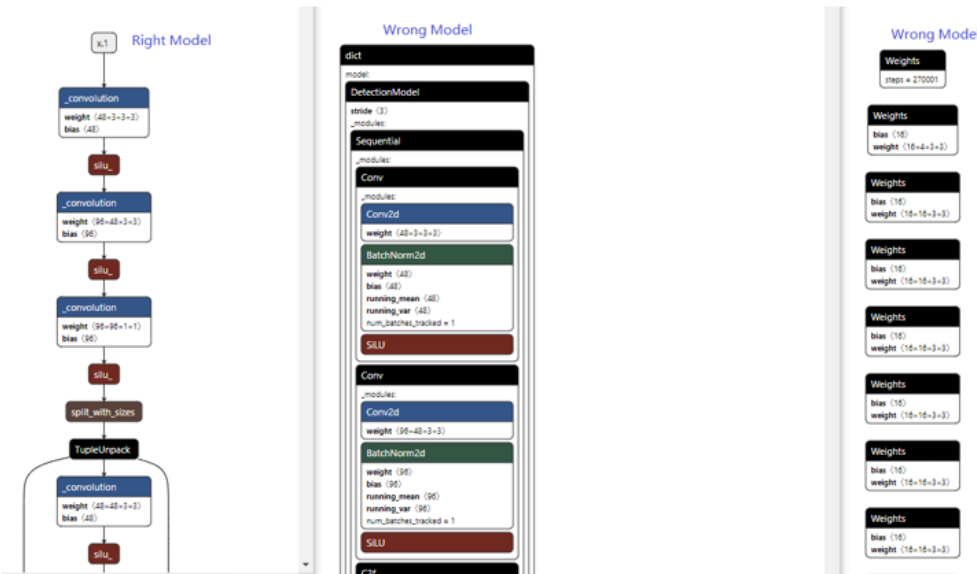
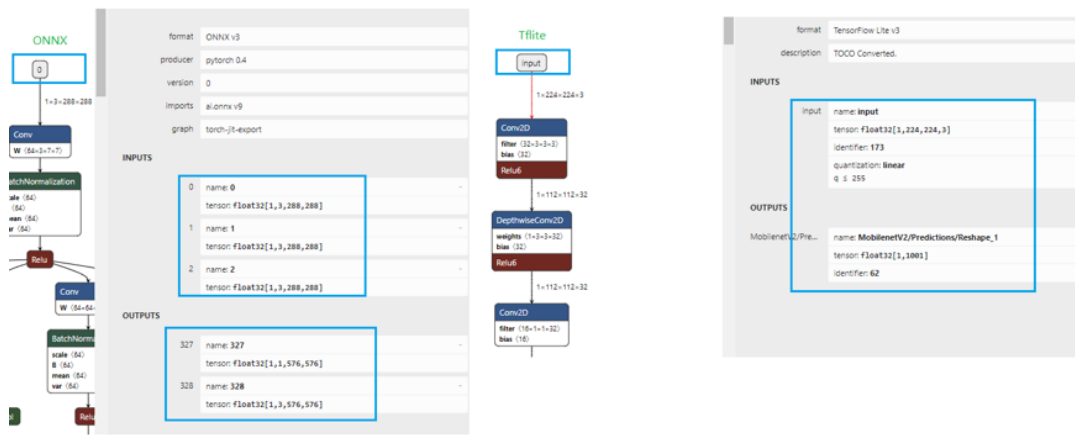


图3.2-模型可视化

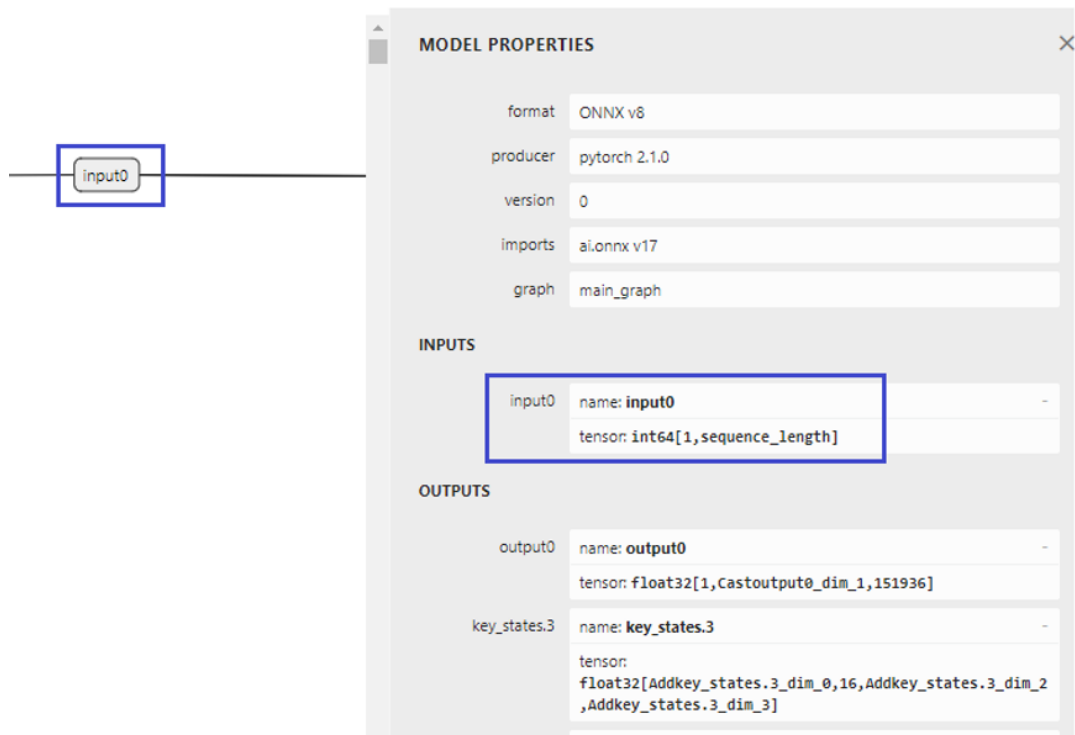
- 模型输入信息

- Onnx/Tflite模型有固定shape



○ Onnx模型无固定shape

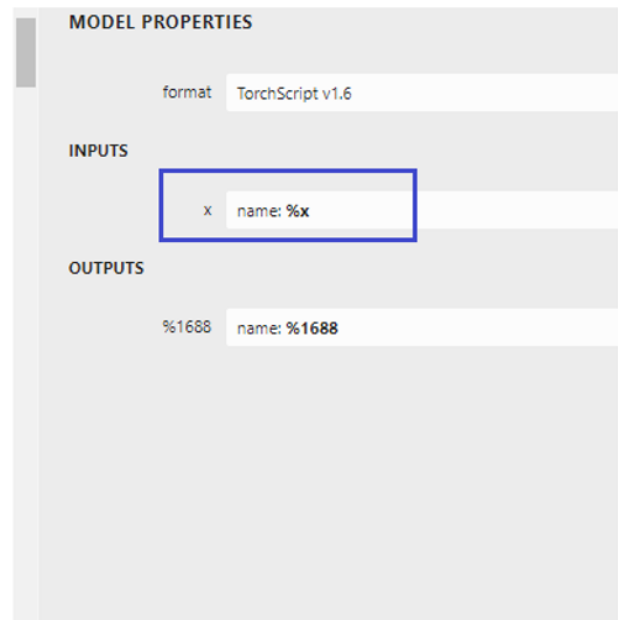
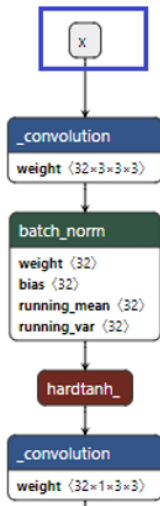
- 模型转换时，需显式设置inputs/input_shapes等参数。



○ Pytorch 模型没有固定Shape

- 模型转换时，需要显式设置inputs/input_shapes/dtypes等参数信息。

pytorch mobilenetv2



3.2.2量化数据集准备

当前amlnn_toolkit支持两种提供量化数据集的方式：1：图片数据集 2：NPY数据集。图片数据集只适用于模型训练时原始数据来源为图片的模型。其他情况，均建议使用NPY数据集。

• 为什么需要数据集

目前，A311Y3 和 C305X2 两个平台支持全部转换类型，其余平台仅支持 W8A8 和 W8A16 的定点运算。将开源框架模型转换为定点模型时，需要进行浮点到定点的量化转换——即把推理过程中每个节点的 float min-max 映射到定点取值范围，因此需要准备量化数据集。量化数据集是多组 多维数组，用于模型量化训练或精度验证。

• 图片数据集

使用图片数据集时，需要确认模型前处理的normalization_mean和normalization_std信息。根据模型训练时的预处理信息来设置参数。对于读取原始图片后的三通道数据（data1，data2，data3），range（0~255），预处理流程为：

```

channel1 = (data1-normalization_mean[0])/normalization_std[0]
channel2 = (data2-normalization_mean[1])/normalization_std[1]
channel3 = (data3-normalization_mean[2])/normalization_std[2]
  
```

使用图片数据集时，一般需要设置前处理参数。常用前处理参数参考如下：

- 归一化到[-1,1]: `normalization_mean=[[127.5,127.5,127.5]]`, `normalization_std=[[127.5,127.5,127.5]]`
- 归一化到[0,1]: `normalization_mean=[[0,0,0]]`, `normalization_std=[[255,255,255]]`
- 不做前处理: `normalization_mean=[[0,0,0]]`, `normalization_std=[[1,1,1]]` 该场景可以不设置mean和std参数。

对于多输入模型，如果需要对多个输入设置不同的前处理参数，参考如下：

```

normalization_mean=[[127.5,127.5,127.5],[0,0,0]] , normalization_std=
[[127.5,127.5,127.5],[255,255,255]]
  
```

• NPY数据集

适用于不能使用图片数据集的场景，例如：大模型，语音模型或者其他一些非图片输入模型。建议客户在原始工程环境下做完预处理之后，将最后的数据保存成numpy文件。使用numpy数据集时，不需要设置normalization_mean和normalization_std参数。

```
#numpy的保存参考代码
import numpy as np
from PIL import Image

import torchvision.transforms as transforms

def preprocess(path):
    transform = transforms.Compose([ transforms.Resize(256),
                                     transforms.CenterCrop(224),
                                     transforms.ToTensor(),
                                     transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])])
    img = Image.open(path).convert('RGB')
    img = transform(img)
    img=img.transpose([1,2,0])
    img = np.expand_dims(img,axis=0).astype('float32')

    np.save("input_1.npy",img)
```

- 量化数据集整理

量化数据集一般建议提供200~500组，确保量化过程中统计出的最大/最小值为实际运行场景的最大/最小值。数据集在调用anmlnn.compile接口时通过dataset参数设置，normalization_mean和normalization_std参数信息在调用amlnn.config接口时设置。

数据集dataset.txt里面的内容如下：

单输入模型：

```
1.jpg
2.jpg

或者：

1.npy
2.npy
```

多输入模型：（一行放一组输入，中间用空格隔开）

```
input1_1.jpg input2_1.jpg
input1_2.jpg input2_2.jpg

或者：

input1_1.npy input2_1.npy
input1_2.npy input2_2.npy
```

3.3 模型转换

3.3.1 AMLNN初始化以及释放

在使用Amlnn-toolkit的API接口时，需要先调用AMLNN()进行AMLNN对象初始化，结束时调用uninit()方法进行释放。初始化时，通过log_level 来设置log 等级，**默认值：'INFO'**。当前支持['DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL']。DEBUG等级会打印更多的转换信息。

```
#Example

from amlnn.api import AMLNN

amlnn = AMLNN(log_level='INFO')

...

amlnn.uninit()
```

3.3.2 模型加载

amlnn_toolkit目前支持onnx、Tflite(float tflite和quant tflite)、Pytorch(pt 和pt2)等模型的加载转换。不同framework模型需调用对应接口，对应接口调用方法如下：

- **onnx模型加载**

- 使用amlnn.load_onnx()加载onnx模型
- 如果onnx模型的输入shape未固定，需要显式设置inputs和input_shapes参数
- 支持转换过程中对onnx模型做截断，通过outputs参数设置截断node对应的输出tensor name

```
#Example1:
amlnn.load_onnx(model='./yolov8s_sim.onnx')

#Example2: 加载模型，并做截断
amlnn.load_onnx(model='./yolov8s_sim.onnx', \
                outputs=
['/model.22/Concat_output_0', '/model.22/Concat_1_output_0', '/model.22/Concat_2_output_0'])
```

- **pytorch模型加载**

- 使用amlnn.load_pytorch()加载pytorch模型, 模型必须是torchscript格式的。
- torchscript格式模型一般没有输入的shape信息，所以需要通过inputs、input_shapes、dtypes等参数来设置输入相关信息。
- 如果模型输入都是float类型，可以不设置dtypes参数，使用默认值即可。多输入设置方式参考：dtypes=['float32', 'int32']

```
#Example:
amlnn.load_pytorch(model='resnet18.pt',
                  inputs=['%x.2'],
                  input_shapes=[[1, 3, 224, 224]],)
```

- **pt2模型加载**

- 使用amlnn.load_pt2()加载pytorch pt2模型，模型必须是ExportedProgram格式的。

- 支持转换过程中对pt2模型做截断，通过outputs参数设置截断node对应的输出tensor name。

```
#Example:
amlnn.load_pt2(model='yolov8n_exported.pt2')
```

• tflite模型加载

- 使用amlnn.load_tflite()加载tflite模型。
- 支持tflite的bypass导入并转换出adla，bypass通过参数quantized_model=True来控制。
- 本身就是量化tflite模型，如w4a8/w8a8等模型时，需要显式设置quantized_model=True，否则会转换出错。本身是float tflite模型，设置quantized_model=True，会转成f16*f16.adla。

```
#Example1:
amlnn.load_tflite(model='yolov8n_sim.tflite')

#Example2:
amlnn.load_tflite(model='yolov8n_sim_w8a8.tflite',quantized_model=True)
```

• Tensorflow模型加载 (TODO)

- 使用amlnn.load_tensorflow()加载tensorflow模型

• Paddle模型加载 (TODO)

- 使用amlnn.load_paddle()加载paddle模型

• Keras模型加载 (TODO)

- 使用amlnn.load_keras()加载keras模型

3.3.3 模型转换参数配置

模型编译转换之前，需要通过amlnn.config接口设置模型转换相关参数，确保转换结果的正确性。通用配置参数如下：

- **target_platform**：指定生成对应平台类型的ADLA文件，根据2.1.3章节：平台类型确认，获取正确的值。当前支持类型如下：

Part_number	target_platform
C308L / C302X	PRODUCT_PID0XA001
S928X	PRODUCT_PID0XA002
A311D2	PRODUCT_PID0XA003
T968D4	PRODUCT_PID0XA004
S905X5 / S905D5	PRODUCT_PID0XA005
C302X2	PRODUCT_PID0XA006
A311Y3	PRODUCT_PID0XA007
C305X2/A123X	PRODUCT_PID0XA008

- **quantized_dtype**: 指定量化类型，不同平台支持的量化类型种类不同。**默认值**: 'w8a8'，目前支持: ['w4a8','w8a8','w4a16','w8a16','w16a16']
 - 只有PRODUCT_PID0XA007/PRODUCT_PID0XA008 平台支持所有量化类型。
 - 其他平台仅支持:'w8a8' 和 'w8a16'
- **activation_dtype**: 指定激活类型。仅在**quantized_dtype='w8a16'**时生效，**默认值**: 'f16'，目前支持['f16','b16','i16']。
 - 通过activation_dtype来指定转换类型为int8 * int16或者int8 * f16或者int8*bf16。

target_platform	Weights * Feature	Support Params
PRODUCT_PID0XA001 ~ PRODUCT_PID0XA006	Int8*Int8	quantized_dtype='w8a8'
	Int8*Int16	quantized_dtype='w8a16', activation_dtype='i16'
PRODUCT_PID0XA007/PRODUCT_PID0XA008	Int4*int8	quantized_dtype='w4a8'
	Int8*Int8	quantized_dtype='w8a8'
	Int4*f16	quantized_dtype='w4a16', activation_dtype='f16'
	Int4*bf16	quantized_dtype='w4a16', activation_dtype='bf16'
	Int8*int16	quantized_dtype='w8a16', activation_dtype='i16'
	Int8*f16	quantized_dtype='w8a16', activation_dtype='f16'
	Int8*bf16	quantized_dtype='w8a16', activation_dtype='bf16'
	f16*f16	quantized_dtype='w16a16', activation_dtype='f16'
	bf16*bf16	quantized_dtype='w16a16', activation_dtype='bf16'

- **quantized_method**:指定量化方式。**默认值**: 'perchannel'，支持['perlayer', 'perchannel']。perchannel 表示卷积核的每一个通道都有不同的缩放因子。perlayer表示卷积核共用一个缩放因子。perchannel的精度会更好。
- **group_size**: 子通道分组(Sub-channel-wise)量化，即将全连接层的通道划分为更小的子组，以实现更细粒度的精度控制。**默认值**: 32，目前支持: [32,64,128]。
 - 仅在quantized_dtype为W4A8、w4a16时生效。
 - group_size 越小，量化参数越多，精度越好。group_size越大，量化参数越少，性能越好。
- **export_intermediate**: 指定是否将转换中间IR导出成tflite模型文件。**默认值**: False

#Example:

#对应的转换参数为:

#target_platform: PRODUCT_PID0XA007 (A311Y3) quantized_dtype: w8a8
quantized_method: perchannel

amlnn.config(target_platform='PRODUCT_PID0XA007')

- 当前支持的量化转换及对应的 `weight_quant_algo`、`activation_quant_algo` 如下表所示。详细量化转换指导请参考后续输出文档：*NPU Quantization Guide.pdf*

Format (weights*Feature)	quantized_dtype	activation_dtype	weight_quant_algo	activation_quant_algo
Int4*int8	w4a8	None	normal/decoupleQ/lsq	normal/histogram/omse/percentile
Int8*int8	w8a8	None	normal/lsq	normal/histogram/omse/percentile
Int4*f16	w4a16	f16	normal/decoupleQ/lsq	/
Int4*bf16	w4a16	bf16	normal/decoupleQ/lsq	/
Int8*int16	w8a16	i16	normal/lsq	normal/histogram/omse/percentile
Int8*f16	w8a16	f16	normal/lsq	/
Int8*bf16	w8a16	bf16	normal/lsq	/
f16*f16	w16a16	f16	/	/
bf16*bf16	w16a16	bf16	/	/

3.3.4 模型编译

用户加载模型、配置模型参数之后，需要通过`amlnn.compile()`进行模型量化转换。对应参数如下：

- **dataset**：量化校准数据集，通过 `dataset.txt` 指定路径（文件中提供图片或 npy 路径）：
 - **默认值：None**，使用随机数量化，仅供性能测试，不保证精度。此时 config 接口中配置的量化算法等参数失效，默认切换为 min-max 量化。
 - `dataset.txt` 使用 npy 文件时，需完成前处理后再保存为 npy，且 shape 须与 `input-shapes` 参数保持一致。config 中配置的 `mean` / `std` 参数不生效
 - 数据集的准备参考3.2.2章节。
- **iterations**：指定量化所用的数据样本数（图片或 npy 文件），从 `dataset.txt` 中读取。**默认值：-1**，表示遍历全部数据；当 `dataset` 为 `None` 时，仅生成一组随机数据用于量化。

#Example:

```
amlnn.compile(dataset='dataset.txt', iterations=200)
```

3.3.5 导出adla模型

用户通过`amlnn.compile()`进行量化转换后，通过`amlnn.export_adla()`导出adla模型文件。

#Example:

```
amlnn.export_adla()
```

3.3.6 模型转换示例

#Example:

```
from amlnn.api import AMLNN

# 初始化amlnn
amlnn = AMLNN(log_level='INFO')

#加载onnx模型
amlnn.load_onnx(model='resnet34-v1-7.onnx')

#配置模型转换参数
```

```
amlnn.config(normalization_mean=[[127.5,127.5,127.5]],normalization_std=
[[127.5,127.5,127.5]],
              quantized_dtype='w8a8', target_platform='PRODUCT_PID0XA007')

#模型编译
amlnn.compile(dataset='./dataset.txt')

#导出adla模型文件，执行成功后，在resnet34-v1-7_adla_unify目录下生成对应的adla文件。
amlnn.export_adla()
```

3.4 模型评估

3.4.1 模型推理

完成 ADLA 模型转换后，可通过**连板推理**或**板端直接推理**进行模型验证与性能评估。若开发环境仅有 Debian/Yocto 边缘设备，则只能使用**板端直接推理**；若为 host+devices 架构（host 为 Linux X86/Arm PC，devices 为带 Amlogic NPU 的边缘设备），则使用**连板推理**，但当 devices 为 Debian/Yocto 边缘设备时，也可使用**板端直接推理**。两种推理方式介绍如下：

方式一：板端直接推理

- 板端安装 amlnn_edge_toolkit（或 lite 版）whl 包。
- 基于 amlnn_edge_toolkit（或 lite 版），通过 Python API 完成板端推理。

方式二：连板推理

host 端安装 amlnn_toolkit（或 amlnn_edge_toolkit），通过 ADB 协议连接板端并传输数据，完成推理。该方式依赖 nn_server 服务，板端需提前启动。

Step1: 获取nnserver文件:

获取nnserver文件,link:https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/nnserver.根据[2.1.2 系统环境确认]确认的系统环境，选取对应的nnserver文件。

- **Android 64-bit:** amlnn_toolkit/nnserver/android/lib64/nnserver
- **Android 32-bit:** amlnn_toolkit/nnserver/android/lib32/nnserver
- **Armbian/Yocto 64-bit:** amlnn_toolkit/nnserver/linux/yocto/lib64/nnserver
- **Armbian/Yocto 32-bit:** amlnn_toolkit/nnserver/linux/yocto/lib32/nnserver
- **Buildroot 64-bit:** amlnn_toolkit/nnserver/linux/buildroot/lib64/nnserver
- **Buildroot 32-bit:** amlnn_toolkit/nnserver/linux/buildroot/lib32/nnserver

Step2: 启动nnserver:

```
#Push the nnserver file to the board and start it.
```

```
adb shell "mkdir -p /data/nn"
adb push /path/to/nnserver /data/nn/
adb shell "chmod +x /data/nn/nnserver"
adb shell "cd /data/nn && ./nnserver"
```

```
# If information similar to the following is displayed, the nnserver service has
started successfully
NNSERVER, vx.x.x, 20xx.xx
Coordinator listening on port 8307
Server listening on port 8308
Server listening on port 8309
Server listening on port 8310
```

模型推理阶段使用的主要接口如下：

- 初始化运行环境接口 `amlnn.init_runtime()` 参数如下：
 - **mode**: 运行模式, 支持 `native` 和 `'nnserver'`。lite版本仅支持 `'native'`。
 - **board_work_path**: 板端工作目录,保存一些运行过程中的中间文件, 默认值为 `/data/nn`。
 - **enable_perf**: 是否开启性能采集功能, 默认 `False`; 如需获取性能信息必须设置为 `True`。
- 加载模型接口 `amlnn.load_model()` 参数如下：
 - **path**: ADLA 模型路径。若已通过 `amlnn_toolkit` (或 `amlnn_edge_toolkit`) 调用 `amlnn.export_adla` / `amlnn.init_runtime`, 则无需再显式调用该接口。
- 推理接口 `amlnn.inference()` 参数如下：
 - **inputs**: 输入数据, 可传单个 `numpy.ndarray`, 也可传 `list` / `tuple`。
 - **run_cycles**: 推理循环次数, 默认为1。

```
import argparse
import numpy as np

from amlnn.api import AMLNN

def preprocess(image_path):
    pass

def postprocess(outputs):
    pass

# init amlnn
amlnn = AMLNN(log_level='INFO')

#load onnx model
amlnn.load_onnx(model='resnet34-v1-7.onnx')

#配置模型转换参数
amlnn.config(normalization_mean=[[127.5,127.5,127.5]],normalization_std=
[[127.5,127.5,127.5]],
quantized_dtype='w8a8', target_platform='PRODUCT_PID0XA007')

#compile model
```

```

amlnn.compile(dataset='./dataset.txt')

#export adla model file
amlnn.export_adla()

# Initialize environment configuration
ret = amlnn.init_runtime(mode='native',enable_perf=True)
if ret != 0:
    raise RuntimeError("init_runtime failed")

# Get sdk version
sdk_version = amlnn.get_sdk_version()
print(sdk_version)

# Get tensor info
tensor_info = amlnn.get_tensor_info()
print(tensor_info)

input_data = preprocess(image_path)

# Set input & Run inference & Get output
outputs = amlnn.inference(inputs=[input_data])

postprocess(outputs)

# Release resources
amlnn.uninit()

```

3.4.2 模型性能评估

调用 `amlnn.get_perf_info()` 可获取模型最近一次推理的性能信息，包括 NPU 推理耗时、帧率、推理占用内存、带宽、硬件/软件算子总耗时等。使用该接口需满足两个前提：

- `amlnn.init_runtime()` 时 `enable_perf=True`
- 已调用 `amlnn.inference()`

调用`amlnn.get_perf_info()`接口后，返回值为 `str`，格式化后的性能信息字符串，字段包括：

- `npu_avg_time_ms`：NPU 平均推理耗时，单位为毫秒；当 `run_cycles > 1` 时返回多轮平均值。
- `fps`：基于 `npu_avg_time_ms` 计算得到的帧率。
- `memory_usage_mb`：当前模型推理所需内存占用，单位为 MB。
- `bandwidth_mb`：当前模型推理涉及的总带宽读写量，单位为 MB。
- `elapsed_in_hw_op_ms`：硬件算子总耗时，单位为毫秒。
- `elapsed_in_sw_op_ms`：软件算子总耗时，单位为毫秒。

```

#example

# init amlnn
amlnn = AMLNN(log_level='INFO')

#load onnx model
amlnn.load_onnx(model='resnet34-v1-7.onnx')

```

```

amlnn.config(normalization_mean=[[127.5,127.5,127.5]],normalization_std=
[[127.5,127.5,127.5]],
quantized_dtype='w8a8', target_platform='PRODUCT_PID0XA007')

#compile model
amlnn.compile(dataset='./dataset.txt')

#export adla model file
amlnn.export_adla()

# Initialize environment configuration
ret = amlnn.init_runtime(model='native',enable_perf=True)

# Set input & Run inference & Get output
outputs = amlnn.inference(inputs=[input_data])

perf_info = amlnn.get_perf_info()
print(perf_info)

#返回值:
# npu_avg_time_ms=9.624, fps=103.907, memory_usage_mb=5.180, bandwidth_mb=28.802,
elapsed_in_hw_op_ms=5.499, elapsed_in_sw_op_ms=0.000

```

3.4.3 模型精度评估

当前仅支持在推理后的 postprocess 中添加后处理逻辑，验证模型精度。后续将支持 ADLA 模型及中间 IR（可导出为 TFLite）逐层精度对比。

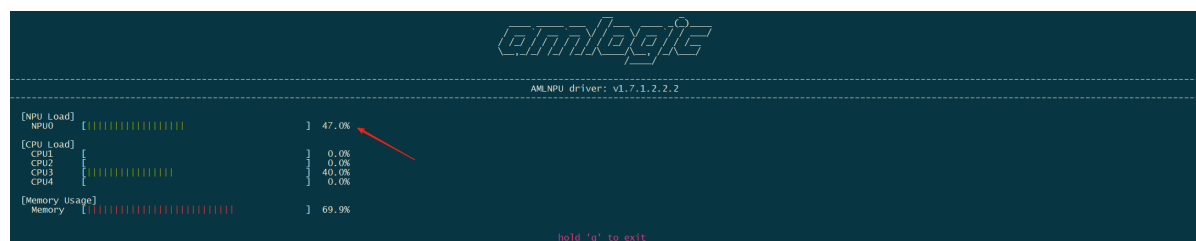
3.4.4 NPU 利用率

查看 npu 利用率的脚本为 NPU_utilization.py，在 demo 的运行过程中，再新开一个终端，执行如下命令即可实时查看 cpu 与 npu 的利用率。

工具link: https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/function_tool/01_adla_get_utilization_tool

```
python NPU_utilization.py
```

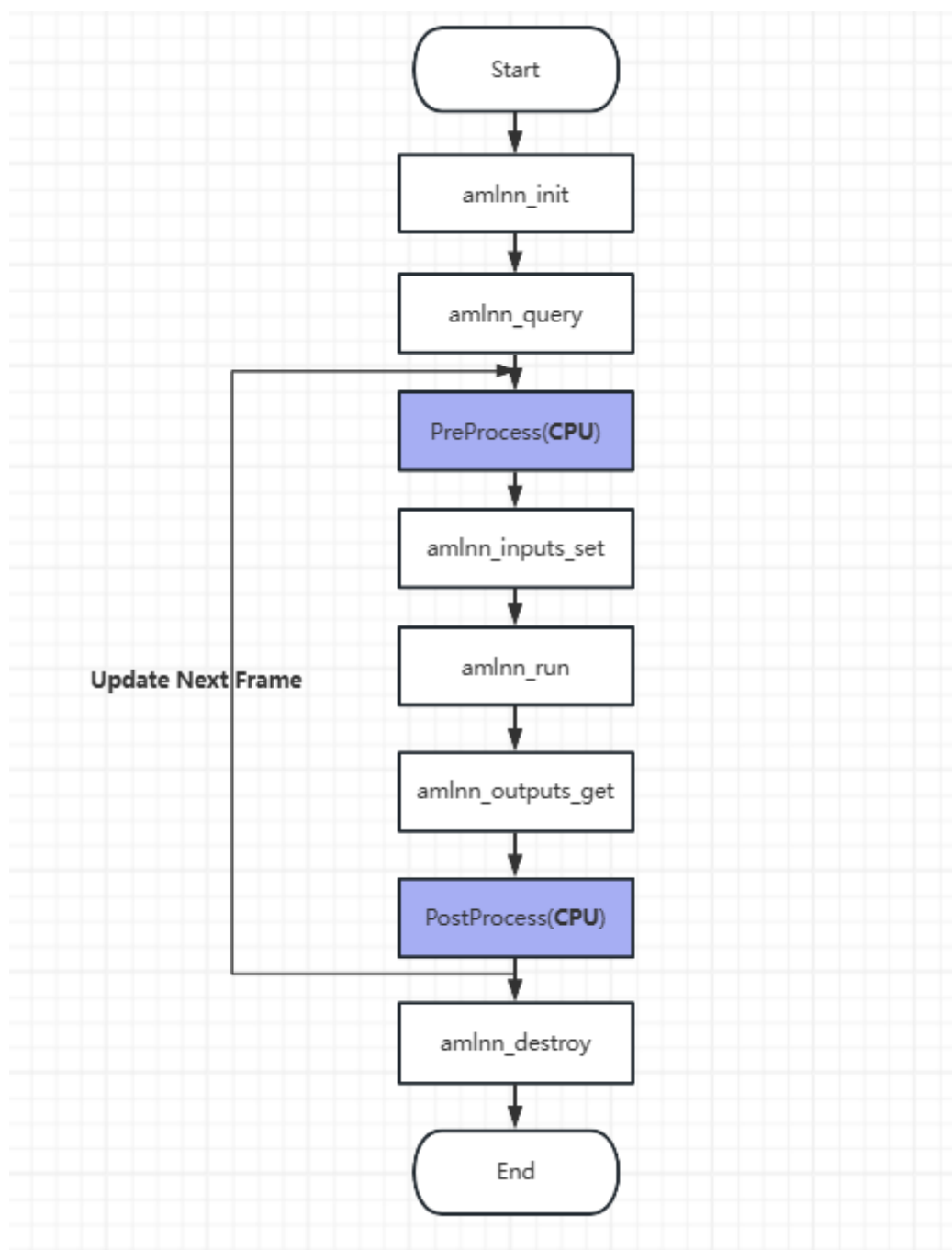
效果如下图所示：



该脚本以 50ms 为间隔读取设备端 NPU 利用率节点。仅当采样时刻恰好落在模型 NPU 推理窗口内，才会显示非零利用率。对于推理耗时较短的模型（如 MobileNet，约 2ms），多数采样点将显示为 0。若需更直观地观察 NPU 真实利用率，可调大 demo 脚本的 `--run-cycles` 参数，使模型连续推理多次，延长推理窗口，便于采样捕获。

3.5 板端C API推理

3.5.1 流程简介



通用 API 调用流程示例代码如下。详细接口使用参考：[NPU API Reference NNSDK2.pdf](#)

```
void *ctx = NULL;
amlnn_init_config init_config;
memset(&init_config, 0, sizeof(amlnn_init_config));

//init
ret = amlnn_init(&ctx, (void *)model_path, 0, &init_config);

// get input output number
amlnn_input_output_num io_num;
ret = amlnn_query(ctx, AMLNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
printf("model input num: %d, output num: %d\n", io_num.n_input,
io_num.n_output);
```

```

// get input attr
amlnn_tensor_attr input_attrs[io_num.n_input];
memset(input_attrs, 0, sizeof(input_attrs));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    input_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_INPUT_ATTR, &(amp;input_attrs[i]),
sizeof(amlnn_tensor_attr));
}

// get output attr
amlnn_tensor_attr output_attrs[io_num.n_output];
memset(output_attrs, 0, sizeof(output_attrs));
for (uint32_t i = 0; i < io_num.n_output; i++)
{
    output_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_OUTPUT_ATTR, amp;output_attrs[i]),
sizeof(amlnn_tensor_attr));
}

amlnn_input inputs[io_num.n_input];
amlnn_output outputs[io_num.n_output];
memset(inputs, 0, sizeof(inputs));
memset(outputs, 0, sizeof(outputs));

// PreProcess
input_data = PreProcess(image_path, xxxx);

//set input
memset(inputs, 0, sizeof(inputs));
inputs[0].index = 0;
inputs[0].buf = input_data; // 需要在外层做好前处理，包括归一化、量化、数据类型匹配，数据布
局格式匹配等，sdk 内部会将该块数据直接拷贝到 npu buf
inputs[0].size = input_size; // 输入数据字节数
ret = amlnn_inputs_set(ctx, io_num.n_input, inputs);

// Run adla
ret = amlnn_run(amlnn_ctx, nullptr);

// Get Output
ret = amlnn_outputs_get(amlnn_ctx, io_num.n_output, outputs, NULL);

// PostProcess
post_process(outputs, results);

amlnn_destroy(ctx);

```

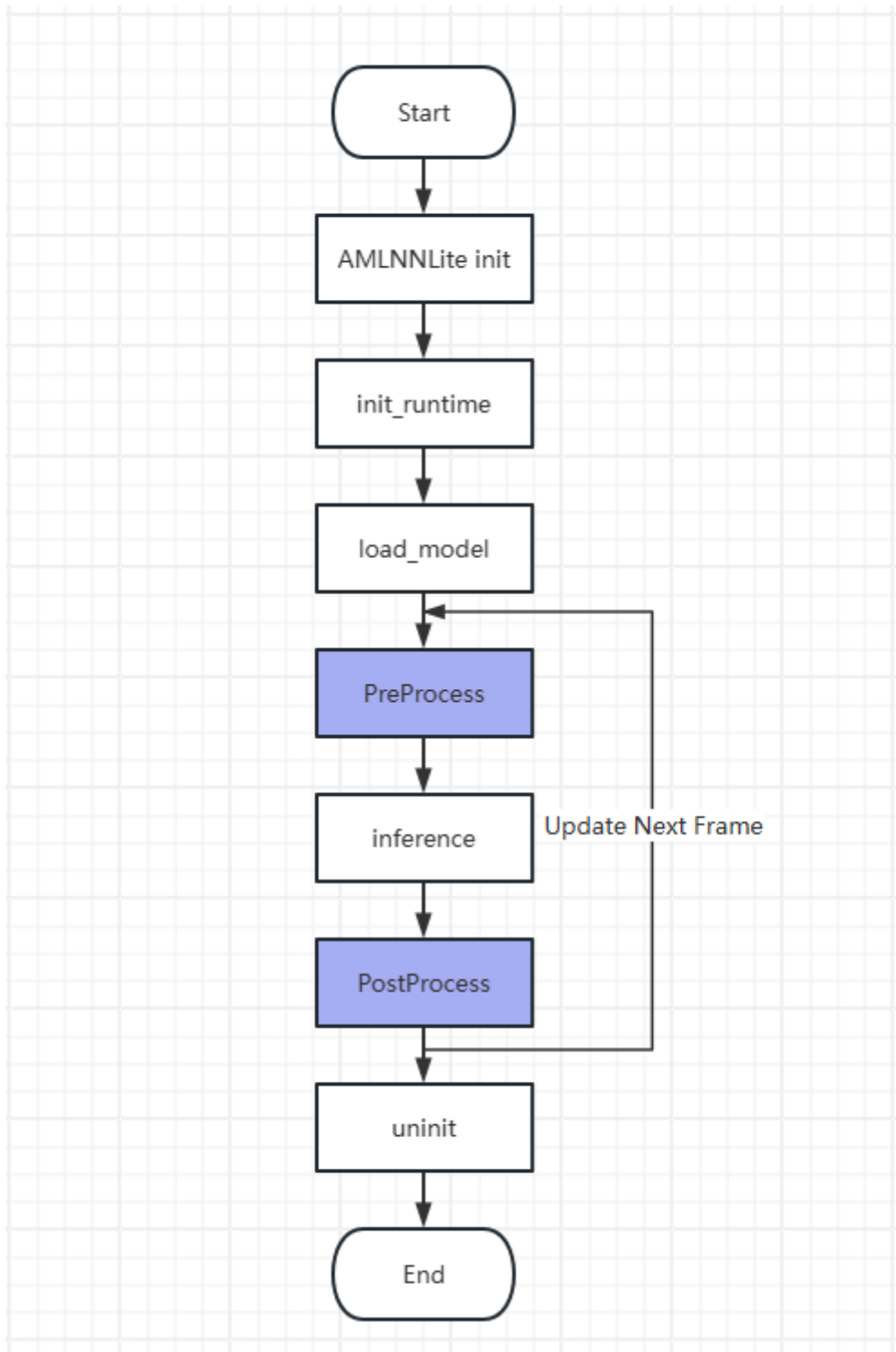
4 板端Python API推理

`amlnn_edge_toolkit_lite` 是 Amlogic 专为板端 Debian/Yocto 系统开发的 Python 推理 whl 包，方便用户在边缘设备上基于 Python 进行 AI demo 开发。它仅包含 `amlnn_edge_toolkit` 的推理部分，只支持 native 方式在板端直接推理，接口与 `amlnn_edge_toolkit` 基本一致。使用需求如下：

- 板端 Python API，要求板端环境为 Debian/Yocto 系统
- 需先通过 `amlnn_toolkit` 导出 ADLA 文件

4.1 Python API流程简介

所有推理相关操作均通过 `AMLNNLite` 类完成。当前 `AMLNNLite` 仅支持板端 `native` 模式。



示例代码如下：

```
from amlnnlite.api import AMLNNLite

#init
amlnnlite = AMLNNLite()

#init runtime
amlnnlite.init_runtime()

#load adla
amlnnlite.load_model(path="xxx.adla")

#preprocess
input_data=PreProcess()

#inference
outputs = amlnnlite.inference(inputs=[input_data])

#PostProcess
PostProcess(outputs)

#uninit
amlnnlite.uninit()
```

4.2 Python API说明

4.2.1 AMLNNLite初始化

在使用Amlnn-toolkit-lite的API接口时，需要先调用AMLNNLite()进行AMLNN对象初始化，结束时调用uninit()方法进行释放。

功能：

- 创建一个 AMLNNLite 实例
- 设置 Python 侧日志等级

参数名	类型	说明
log_level	str	日志等级，支持 DEBUG / INFO / WARNING / ERROR

```
from amlnnlite.api import AMLNNLite

amlnnlite = AMLNNLite(log_level="INFO")
```

4.2.2 init_runtime

功能：

- 初始化运行环境
- 在 AMLNNLite 中，`mode` 仅支持 `native`

返回值：0：成功，-1：失败

参数名	类型	说明	必要参数
mode	str	运行模式，默认值:'native'，lite版本仅支持 <code>native</code> 。	否
board_work_path	str	板端工作目录，默认值为 <code>/data/nn</code> ，存放运行过程中的中间件	否
enable_perf	bool	是否开启性能采集，默认值： <code>False</code> ，如需获取性能信息必须设置为 <code>True</code>	否

```
ret = amlnnlite.init_runtime()
```

4.2.3 get_sdk_version

功能：获取当前板端 SDK / ADLA / DELEGATE 相关版本信息

返回值：`str`，版本字符串

```
sdk_version = amlnnlite.get_sdk_version()
print(sdk_version)
```

4.2.4 load_model

功能：加载runtime运行模型

返回值：0：成功，-1：失败

参数	类型	说明	必要参数
path	str	本地模型路径，目前仅支持 <code>.adla</code>	是

```
#Example
model_info = amlnnlite.load_model(path="./adla/resnet18-v1-7_int8_A311D2.adla")
```

4.2.5 get_tensor_info

功能：获取模型输入输出tensor信息

返回值：

- dict，包含两个字段：
 - `inputs`：模型输入 tensor 信息列表
 - `outputs`：模型输出 tensor 信息列表

```
tensor_info = amlnnlite.get_tensor_info()
print(tensor_info)
```

每个 tensor 信息均为一个 `dict`，字段说明如下：

字段名	类型	说明
index	int	当前 tensor 在输入或输出列表中的索引
name	str	tensor 名称
n_dims	int	tensor 维度个数
dims	tuple	tensor 原生 shape，顺序与 <code>format_name</code> 对应
n_elems	int	tensor 元素总数
size	int	tensor 原始有效数据大小，单位为字节
size_with_stride	int	对齐后的 buffer 大小，单位为字节
format	int	tensor 布局格式的内部枚举值
format_name	str	tensor 布局格式名称，如 <code>NHWC</code> / <code>NCHW</code>
type	int	tensor 数据类型的内部枚举值
type_name	str	tensor 数据类型名称，如 <code>INT8</code> 、 <code>UINT8</code> 、 <code>FP32</code> 等
zp	int	量化 zero point
scale	float	量化 scale

4.2.6 inference

功能：执行模型推理

返回值：返回输出 tensor 列表

- `list[np.ndarray]`
- 返回的输出 tensor 数据为 `float32` 格式

参数	类型	说明	必要参数
inputs	list/tuple/np.ndarray	输入数据，可传单个 <code>numpy.ndarray</code> ，也可传 <code>list</code> / <code>tuple</code>	是
inputs_data_format	str	输入数据格式，支持 <code>NHWC</code> / <code>NCHW</code> ，默认值为 <code>NHWC</code>	否

参数	类型	说明	必要参数
outputs_data_format	str	输出数据格式，支持 <code>NHWC</code> / <code>NCHW</code> ，默认值为 <code>NHWC</code>	否
run_cycles	int	推理循环次数，默认值为 1	否

```
outputs = amlnnlite.inference(
    inputs=[input_data],
    inputs_data_format="NHWC",
    outputs_data_format="NHWC",
    run_cycles=1,
)
```

4.2.7 uninit

功能：释放运行时与模型资源

```
amlnnlite.uninit()
```

4.2.8 Python API 示例Demo

```
import argparse
import numpy as np

from amlnnlite.api import AMLNNLite

def preprocess(image_path):
    pass

def postprocess(outputs):
    pass

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--model-path", default="./model_file/xxx.adla")
    parser.add_argument("--image-path", default="./input/xxx.jpg")
    args = parser.parse_args()

    # Initialize AMLNNLite
    amlnnlite = AMLNNLite(loglevel="ERROR")

    # Initialize environment configuration
    ret = amlnnlite.init_runtime(enable_perf=False)
    if ret != 0:
        raise RuntimeError("init_runtime failed")

    # Get sdk version
    sdk_version = amlnnlite.get_sdk_version()
```

```
print(sdk_version)

# Load model
amlnnlite.load_model(path=args.model_path)

# Get tensor info
tensor_info = amlnnlite.get_tensor_info()
print(tensor_info)

input_data = preprocess(args.image_path)
# Set input & Run inference & Get output
outputs = amlnnlite.inference(inputs=[input_data])

postprocess(outputs)
# Release resources
amlnnlite.uninit()

if __name__ == "__main__":
    main()
```